

UNIVERSIDAD TECNOLÓGICA DE TEHUACÁN

Nombre del alumno:

ANGEL GASPAR REYES

8A

TSU EN TECNOLOGIAS DE LA
INFORMACION: DESARROLLO DE
SOFTWARE Y MULTIPLATAFORMA

TEHUACÁN, PUEBLA, 8 de Marzo de 2026

Arquitectura de Autenticación y Control de Roles

1. ¿Qué es autenticación y en qué se diferencia de autorización?

Autenticación (Authentication)

La **autenticación** es el proceso mediante el cual un sistema **verifica la identidad de un usuario**.

Normalmente se realiza mediante:

- Usuario y contraseña
- Token de acceso
- Biometría
- Código enviado al teléfono o correo

Ejemplo práctico:

Un usuario ingresa:

correo: angel@email.com

contraseña: *****

El sistema valida los datos en la base de datos y confirma que ese usuario existe.

Autorización (Authorization)

La **autorización** ocurre **después de la autenticación** y determina:

¿Qué puedes hacer dentro del sistema?

Aquí se controlan **permisos y roles**.

Usuario	Rol	Permiso
Ángel	Administrador	Crear usuarios
Carlos	Vendedor	Registrar ventas
María	Cliente	Ver pedidos

Aunque todos estén autenticados, **no todos pueden hacer lo mismo**.

2. ¿Qué son JWT, sesiones y OAuth?

JWT (JSON Web Token)

Un **JWT** es un **token que contiene información del usuario firmada digitalmente**.

Cuando un usuario inicia sesión:

1. El servidor valida credenciales
2. Genera un token
3. El cliente guarda ese token
4. El token se envía en cada petición

Ejemplo de flujo:

Login → servidor valida → genera JWT → cliente guarda token

Luego:

Petición → token incluido → servidor valida token → acceso permitido

Ventajas:

- No requiere guardar sesiones en el servidor
- Escala bien
- Muy usado en **APIs REST**

Desventajas:

- Si el token es robado, puede ser usado hasta que expire

Sesiones (Sessions)

Las **sesiones** almacenan información del usuario **en el servidor**.

Flujo:

1. Usuario inicia sesión
2. El servidor crea una sesión
3. El navegador guarda un **Session ID en cookies**
4. Cada petición incluye esa cookie

Ejemplo:

Usuario → login

Servidor → crea sesión

Cliente → guarda cookie

Ventajas:

- Mayor control desde el servidor
- Fácil de invalidar

Desventajas:

- Consume memoria del servidor
- Escala peor en sistemas muy grandes

OAuth

OAuth es un **protocolo de autorización** que permite iniciar sesión con cuentas externas.

Ejemplos conocidos:

- Iniciar sesión con Google
- Iniciar sesión con GitHub
- Iniciar sesión con Facebook

Flujo simplificado:

Usuario → "Login con Google"

Sistema → redirige a Google

Google → autentica usuario

Google → devuelve token al sistema

Ventajas:

- No manejas contraseñas
- Seguridad delegada

Desventajas:

- Dependencia de terceros

3. ¿Cuándo conviene usar tokens y cuándo sesiones?

Usar JWT cuando:

- El sistema usa **APIs REST**
- Hay **aplicaciones móviles**
- Hay **microservicios**
- Se necesita escalabilidad

Ejemplo:

- Apps móviles
- Sistemas tipo SaaS
- Arquitecturas distribuidas

Usar sesiones cuando:

- Es una **aplicación web tradicional**
- No hay alta escalabilidad
- Se quiere control total del login

Ejemplo:

- Panel administrativo
- Sistemas internos de empresa

4. Riesgos de seguridad en un mal diseño de autenticación

Un sistema mal diseñado puede sufrir ataques como:

Robo de tokens

Si un atacante obtiene un token válido puede acceder al sistema.

Solución:

- HTTPS obligatorio
- Tokens con expiración corta

Fuerza bruta

Intentar miles de contraseñas hasta acertar.

Solución:

- Limitar intentos de login
- Captcha
- Bloqueo temporal de cuenta

Escalación de privilegios

Un usuario normal logra permisos de administrador.

Solución:

- Validar roles en **cada petición**
- No confiar en datos del frontend

Inyección o manipulación de tokens

Un atacante intenta modificar el token.

Solución:

- Firmar tokens con claves seguras
- Validar firma siempre

5. ¿Cómo se definen roles y permisos en sistemas reales?

Los sistemas profesionales utilizan **RBAC (Role Based Access Control)**.

Esto significa que **los permisos se asignan a roles**, no directamente a usuarios.

Ejemplo de estructura

Usuarios

id	nombre	rol
1	Ángel	admin
2	Luis	vendedor
3	Ana	cliente

Roles

Rol	Descripción
admin	control total
vendedor	gestiona ventas
cliente	consulta información

Permisos

Permiso
crear_usuario
eliminar_usuario
registrar_venta
ver_productos

Relación Rol → Permisos

Rol	Permiso
admin	todos
vendedor	registrar_venta
cliente	ver_productos

Diseño del flujo de autenticación

Un flujo típico sería:

Usuario abre aplicación



Pantalla de login



Ingresa correo y contraseña



Servidor valida credenciales



Servidor genera JWT



Cliente guarda token



Cliente envía token en cada petición



Servidor valida token



Servidor verifica rol



Acceso permitido o denegado

Ejemplo de roles para un sistema

Supongamos un sistema empresarial:

Tipos de usuario

1. **Administrador**
2. **Empleado**
3. **Cliente**

Permisos

Administrador:

- crear usuarios
- editar usuarios
- ver reportes
- administrar sistema

Empleado:

- registrar operaciones
- ver clientes

Cliente:

- ver información
- actualizar perfil

Riesgos que este diseño mitiga

Este enfoque permite prevenir:

- acceso no autorizado
- manipulación de permisos
- uso indebido de cuentas
- escalación de privilegios

Ejemplo de Issues para implementación futura

Issue 1

Implementar endpoint de login

POST /auth/login

Issue 2

Generar JWT tras autenticación

Issue 3

Middleware para validar token

Issue 4

Middleware de control de roles

Issue 5

Endpoint para refrescar token

1 Diagrama de autenticación

Debe mostrar:

Usuario



Login



Servidor



Validación



Generación Token



Acceso a recursos



Validación de rol

2 Documento técnico

Debe explicar:

- arquitectura elegida
- método de autenticación
- control de roles
- riesgos mitigados

3 Issues para implementación

Lista de tareas para programar el sistema después.